サンプル問題

Ｐｙｔｈｏｎプログラミング能力認定試験

１　級

解答時における注意事項

１.次の表に従って解答してください。

問題番号	問１～問７
選択方法	全問必須
試験時間	９０分

２.HB の黒鉛筆を使用してください。訂正する場合は，あとが残らないように消しゴムできれいに消し，消しくずを残さないでください。なお，ボールペンや万年筆等で記入した場合は，採点されません。

３.マークシート（解答用紙）の所定の欄に，級種，会場コード，受験番号を記入しマークしてください。また，会場名，氏名及びフリガナを所定の位置に記入してください。

４.解答は，次の例題にならって，「解答マーク欄」にマークしてください。

〔例題〕　日本の首都はどこか。

ア　東京　　イ　京都　　ウ　大阪　　エ　福岡

正しい答えは“ア　東京”ですから，次のようにマークしてください。

例題　■　イ　ウ　エ

指示があるまで開いてはいけません。
試験終了後，問題冊子を回収します。

受験会場	
受験番号	
氏　　名	

サーティファイ
情報処理能力認定委員会

問１～問７は，すべて必須問題です。全問について解答してください。

各設問の答えは，解答群の中から一つだけ選び，括弧中の設問番号に対応したマークシートの解答番号の「解答マーク欄」にマークしてください。なお，二つ以上マークした場合には不正解になります。

問１　Pythonの特徴やオブジェクト指向プログラミングに関する次の記述の正誤を，解答群の中から選べ。

(1)　関数やメソッドをラップすることでその関数やメソッドの前後に処理を追加し，処理内容を変更することができる仕組みを，デコレータという。

(2)　メソッドの引数にキーワード引数を用いると，実行に必要な引数を渡さない場合，あらかじめ指定しておいた引数の値を使ってメソッドが実行される。

(3)　あるクラスを継承して作成したクラスにおいて，継承元のクラスと同名のメソッドや変数を上書きして定義し直すことを，オーバーライドという。

(4)　無名関数では，`lambda`というキーワードを用いて複数行の式で定義することができる。

(5)　作成中のPythonのプログラムファイルにおいて，このファイルとは別で同じディレクトリにあるPythonのプログラムファイルProgA.pyに定義されたクラス`ClassA`をインポートして使用するには，作成中のプログラムに「`import ProgA.ClassA`」と記述すればよい。

(6)　外部からはインスタンス変数のように扱え，クラスの実装では値のチェックを行ったり，他のインスタンス変数などを組み合わせた演算結果などを返したりするなど，メソッドのように記述できる仕組みを，プロパティと呼ぶ。

解答群

ア　正しい　　イ　誤り

問２　次のプログラムおよびクラスやオブジェクトに関する説明を読んで，［　　］に入れる適切な字句を，解答群の中から選べ。

＜プログラム＞

```
1   class BMI:
2       def __init__(self, height, weight):
3           self.height = height
4           self.weight = weight
5
6       def calc_bmi(self):
7           bmi = self.weight / self.height ** 2
8           print(f'BMI: {bmi:.1f}')
9
10
11  class ObesityIndex(BMI):
12      boundary_age = 18
13
14      def __init__(self, height, weight, age):
15          super().__init__(height, weight)
16          self.age = age
17
18      def calc_rohrer(self):
19          rohrer = 10 * self.weight / self.height ** 3
20          print(f'Rohrer 指数: {rohrer:.1f}')
21
22      def calc_index(self):
23          if self.age >= self.boundary_age:
24              self.calc_bmi()
25          else:
26              self.calc_rohrer()
27
28      @classmethod
29      def which_index(cls, age):
30          message = 'BMI を表示します'
31          if age < cls.boundary_age:
32              message = 'Rohrer 指数を表示します'
33          print(message)
```

＜クラスやオブジェクトに関する説明＞

1. BMI クラスの__init__メソッドは，[(7)] する際に実行される特殊メソッドである。[(8)] とも呼ばれ，インスタンス変数の初期化などを行うために使用される。
2. BMI クラスの [(9)] である calc_bmi メソッドの第 1 引数（self）は，[(10)] 。
3. ObesityIndex クラスは BMI クラスを継承しており，継承元のクラスの変数に加えて，[(11)] を新たな変数として加えている。また同様に，継承元のクラスのメソッドに加えて，[(12)] を新たなメソッドとして加えている。
4. @classmethod は，クラスメソッドとして定義するメソッドに付加するもので，which_index メソッドは，[(13)] 。
5. calc_index メソッドや which_index メソッドが参照している boundary_age はクラス変数であり，[(14)] 。

(7)の解答群

ア　インスタンス変数にアクセス
イ　インスタンスを生成
ウ　クラスに定義されたメソッドのうちいずれかを実行
エ　クラスをインポート

(8)の解答群

ア　コンストラクタ
イ　ジェネレータ
ウ　デストラクタ
エ　モジュール

(9)の解答群

ア　インスタンスメソッド
イ　関数
ウ　グローバル変数
エ　スタティックメソッド

(10)の解答群

ア　インスタンス変数にアクセスしない場合，省略することができる
イ　クラス変数にアクセスしない場合，省略することができる。
ウ　省略した場合，エラーが発生する。
エ　省略した場合，スタティックメソッドとして扱われる。

(11)の解答群

ア　age の一つ
イ　age，boundary_age の二つ
ウ　age，boundary_age，message，rohrer の四つ
エ　boundary_age の一つ

(12)の解答群

ア　calc_rohrer の一つ
イ　calc_rohrer, calc_index の二つ
ウ　calc_rohrer, calc_index, which_index の三つ
エ　which_index の一つ

(13)の解答群

ア　インスタンスを生成しなくても，クラスから直接実行できる
イ　インスタンスを生成すれば，クラスから直接実行できる
ウ　クラスから直接実行できるが，インスタンスからは実行できない
エ　クラスから直接実行できるが，クラス内の変数を参照することはできない

(14)の解答群

ア　インスタンスの有無にかかわらず，参照できるのはクラスからのみである
イ　インスタンスを生成していなくても，クラスから参照することができる
ウ　インスタンスを生成すれば，クラスからも参照することができるようになる
エ　クラスから参照することができるのは，インスタンスが生成済みの場合だけである

問３　次のプログラムに対する設問の答えとして適切なものを，解答群の中から選べ。

（15）次のプログラムを実行した結果，「7」が出力された。プログラム中の［　　］に入れる記述として，正しいものはどれか。

```
1   a = 0b110
2   b = 0b010
3   c = [　　]
4   print(c)
```

解答群

ア　a & b >> 1
イ　a & b << 1
ウ　a | b >> 1
エ　a | b << 1

（16）次のプログラムを実行した結果，「[2, 6, 24, 24, 120]」が出力された。プログラム中の［　　］に入れる記述として，正しいものはどれか。

```
1   list_a = []
2   p = 1
3   for i in range(1, 6):
4       p *= i
5       list_a.append(p)
6   list_b = [　　] + [24, 120]
7   print(list_b)
```

解答群

ア　list_a[:3]
イ　list_a[:4]
ウ　list_a[1:3]
エ　list_a[1:4]

(17) 次のプログラムを実行した結果，「[1, 2, 3, 1, 2]」が出力された。プログラム中の［　　］に入れる記述として，正しいものはどれか。

```
1   d = 100
2   list_a = []
3   while d > 0:
4       d //= 2
5       if d > 10:
6           [　　]
7       for i in range(1, d):
8           if i < 4:
9               list_a.append(i)
10          else:
11              break
12  print(list_a)
```

解答群

ア　break　　イ　continue　　ウ　pass　　エ　return

(18) 次のプログラムを実行した結果，「[2, 4, 1, 2, 3, 4]」が出力された。プログラム中の［　　］に入れる記述として，正しいものはどれか。

```
1   list_a = [1, 2, 3, 4]
2   list_b = list_a[:2]
3
4   def func():
5       [　　]
6       list_a = [2, 4, 6, 8]
7       list_b = list_a[:2]
8
9   func()
10  list_c = list_b + list_a
11  print(list_c)
```

解答群

ア　global list_a
イ　global list_a, list_b
ウ　global list_b
エ　何も記述する必要が無い

(19) 次のプログラムを実行した結果，「True False」が出力された。プログラム中の［　　］に入れる記述として，正しいものはどれか。

```
 1  list_a = [0, 2, 4]
 2  id1 = id(list_a)
 3
 4  list_b = [　　]
 5  list_b[0] = 4
 6  list_b[1] = 6
 7  list_b[2] = 8
 8  id2 = id(list_b)
 9
10  list_a = [0, 2, 4]
11  id3 = id(list_a)
12
13  print(id1 == id2, id1 == id3)
```

解答群

ア　[0, 2, 4]

イ　[x for x in range(0, 4, 2)]

ウ　list_a

エ　list_a.copy()

問４　次のプログラムに対する設問の答えとして適切なものを，解答群の中から選べ。

（20）次のプログラムを実行した結果として，正しいものはどれか。

```
1   a = 17
2   b = 3
3   c = round(a / b)
4   d, m = divmod(a, b)
5   e = pow(d, m)
6   print(e - c)
```

解答群

ア　「19」が表示される。
イ　「20」が表示される。
ウ　「26」が表示される。
エ　「27」が表示される。

（21）次のプログラムを実行した結果として，正しいものはどれか。ただし，出力結果は，異なる表示順である場合も，同一のものとみなすものとする。

```
1   pass_kokugo = {'和也', '良子', '浩二', '陽子'}
2   pass_sugaku = {'浩二', '直人', '美香', '和也'}
3   pass_shakai = {'千代', '美香', '浩二', '良子'}
4   koku_su = pass_kokugo & pass_sugaku
5   su_sha = pass_sugaku & pass_shakai
6   print(koku_su ^ su_sha)
```

解答群

ア　「{'和也', '美香'}」が表示される。
イ　「{'浩二'}」が表示される。
ウ　「{'浩二', '和也', '美香'}」が表示される。
エ　「set()」が表示される。

(22) 次のプログラムを実行した結果として，正しいものはどれか。

```
1   u = [2, 4, 6, 8, 10, 12]
2   u1, *u2, u3, u4 = u
3   print(u2)
```

解答群

ア 「[4]」が表示される。
イ 「[4, 6]」が表示される。
ウ 「[4, 6, 8]」が表示される。
エ 「[4, 10, 12]」が表示される。

(23) 次のプログラムを実行した結果として，正しいものはどれか。

```
1   def gen(k):
2       for i in range(k):
3           yield 2 * i
4
5   g = gen(5)
6   next(g)
7   next(g)
8   next(g)
9   sum = 0
10  for j in g:
11      sum += j
12  print(sum)
```

解答群

ア 「8」が表示される。
イ 「14」が表示される。
ウ 「19」が表示される。
エ 「20」が表示される。

(24) 次のプログラムを実行した結果として，正しいものはどれか。

```
 1  a = '123'
 2  b = 456.0
 3  c = None
 4  try:
 5      c = (a + str(b))[10]
 6  except IndexError:
 7      print('Index Error!')
 8  except TypeError:
 9      print('Type Error!')
10  except ValueError:
11      print('Value Error!')
12  finally:
13      print(c)
```

解答群

ア 「123456.0」が表示される。
イ 「Index Error!」，「None」の順に表示される。
ウ 「Type Error!」，「None」の順に表示される。
エ 「Value Error!」，「None」の順に表示される。

問５　次のプログラムを実行したとき，print関数によって表示される値を，解答群の中から選べ。

＜プログラム＞

```
 1  dict_a = {'Apple': 8, 'Banana': 2, 'Grape': 6, 'Persimmon': 9}
 2  list_b = ['Blueberry', 'Cherry', 'Grapefruit', 'Lemon', 'Plum']
 3
 4  print(dict_a['Grape'])                      … (25)
 5
 6  dict_a['Papaya'] = 3
 7
 8  print(dict_a['Apple'] * dict_a['Papaya'])   … (26)
 9
10  dict_b = {}
11  i = -4
12  for v in dict_a:
13      dict_b[v] = i ** 2 - 4
14      i += 2
15
16  print(dict_b['Banana'])                     … (27)
17
18  list_c = [i + int(i / 2) for i in range(5)]
19
20  dict_c = dict(zip(list_b, list_c))
21  print(dict_c['Cherry'])                     … (28)
22
23  dict_d = dict_a | dict_c
24
25  sum = 0
26  for v in dict_d:
27      if v.startswith('P'):
28          sum += dict_d[v]
29  print(sum)                                  … (29)
```

(25) の解答群

ア 2　　イ 6　　ウ 8　　エ 9

(26) の解答群

ア 6　　イ 18　　ウ 24　　エ 27

(27) の解答群

ア -4　　イ 0　　ウ 4　　エ 12

(28) の解答群

ア 1　　イ 3　　ウ 4　　エ 6

(29) の解答群

ア 7　　イ 18　　ウ 20　　エ 35

問６　次のプログラムの説明およびプログラムを読んで，プログラム中の［　　］に入れる適切な字句を，解答群の中から選べ。

＜プログラムの説明＞

料理名とその料理を作るために必要な材料名とその数量の情報が記録された recipes.json と，在庫としてある材料名とその数量の情報が記録された stocks.json を基に，以下の処理を行うプログラムである。recipes.json と stocks.json のデータ構成およびデータは後述する。

- 指定した料理を在庫の材料だけで作れるかどうかを表示する。材料が不足する場合，不足分を表示する。
- 在庫だけで作れる料理を表示する。また，在庫に材料があっても数量が不足する料理も，不足分と併せて表示する（1 種類でも必要な材料が無い料理は表示しない）。

このプログラムでは，まず recipes.json および stocks.json に記録されているデータを読み込み，変数 recipes および stocks にそれぞれ格納する。続いて，judge 関数でカレー 1 人前，豚汁 1 人前，筑前煮 2 人前の料理が作れるかどうかを判定する。最後に suggest 関数で，在庫のみで 2 人前作れる料理を表示する。

＜関数の仕様＞

○関数judge

整数の引数 ninzu で作る分量を指定し，recipe_name で指定された料理が在庫だけで作れるかどうかを判定し，その結果を表示する関数である。show_full が True の場合，在庫に全く無い材料がある場合に不足分も表示する。関数 judge の仕様を，表 1 に示す。

・処理手順

(1)　recipesの中に指定された料理名recipe_nameが無い場合は，レシピが無い旨を表示して関数の処理を終了する。
(2)　recipe_nameの材料のデータを変数zairyouに代入する。zairyouとstocksそれぞれのキーの集合の差集合diffを求める。
(3)　diffが空集合でない，かつshow_fullがFalseであれば，関数の処理を終了する。
(4)　diffが空集合でない，かつshow_fullがTrueであれば，在庫が全く無い材料について不足分を表示した後，在庫はあるが数量が足りない材料の不足分を表示する。このとき，不足分は作る料理の分量に対応した数量とする。
(5)　diffが空集合の場合，材料ごとに，作る料理の分量を考慮した必要な数量を計算し，材料の数量に不足がある場合は，辞書fusokuに不足する数量を追加していく。このfusokuの要素数が0であれば，在庫のみでその料理を作ることができる。要素数が0でない場合，在庫の不足があるため，それぞれの材料の不足分を表示する。

表１　関数 judge の仕様

judge(ninzu, recipe_name, show_full = True)	
引　数	ninzu　　：作る料理の分量(人数) recipe_name　　：作る料理名 show_full = True　：True の場合，在庫が全く無い材料（stocks.json に記載がない材料）がある料理の不足分を表示させる。False の場合，在庫が全くない材料がある料理は表示しない。
戻り値	なし

○関数suggest

整数の引数 ninzu で作る分量を指定し，recipes に格納されている料理名全てに対して，judge 関数を使用して在庫だけで料理を作れるかを判定し，結果を表示する関数である。ただし，在庫に材料が全く無い料理については，表示しないようにする。関数 suggest の仕様を，表２に示す。

表２　関数 suggest の仕様

suggest(ninzu)	
引　数	ninzu：作る料理の分量(人数)
戻り値	なし

＜データの構成と記録されているデータ＞

recipes.jsonとstocks.jsonのデータの構成を，図１に示す。

recipes.json	stocks.json

{
料理名1: {
材料名: 数量,
材料名: 数量,
⋮
},
料理名2: {
材料名: 数量,
材料名: 数量,
⋮
},
⋮
}
	{
	材料名: 数量,
	材料名: 数量,
	⋮
	}

図１　recipes.jsonとstocks.jsonのデータの構成

recipes.json と stocks.json に記録されているデータの例を，図２に示す。なお，stocks.json には在庫があるもののみを記述することとし，数量 0 の材料は設定しないものとする。

recipes.json

```
{
    "カレーライス": {
        "にんじん（本）": 2,
        "じゃがいも（個）": 2,
        "たまねぎ（個）": 1,
        "豚こま肉（g）": 200
    },
    "豚汁": {
        "にんじん（本）": 1,
        "豚バラ肉（g）": 200,
        "ごぼう（本）": 1,
        "さといも（個）": 4
    },
    "筑前煮": {
        "鶏もも肉（枚）": 1,
        "れんこん（本）": 1,
        "にんじん（本）": 1,
        "ごぼう（本）": 1,
        "しいたけ（個）": 3,
        "こんにゃく（個）": 1
    },
    "ハンバーグ": {
        "ひき肉（g）": 100,
        "たまねぎ（個）": 1,
        "牛乳（ml）": 100,
        "パン粉（g）": 20
    }
}
```

stocks.json

```
{
    "にんじん（本）": 1,
    "豚バラ肉（g）": 200,
    "ごぼう（本）": 1,
    "さといも（個）": 4,
    "じゃがいも（個）": 2,
    "たまねぎ（個）": 1,
    "牛乳（ml）": 100,
    "パン粉（g）": 20,
    "ひき肉（g）": 50
}
```

図２　recipes.json と stocks.json に記録されているデータ

≪実行結果例≫

「カレー」のレシピなし

豚汁（1人前）：調理可能

筑前煮（2人前）：在庫なし(*)，数量不足(-)

* 鶏もも肉（枚）: 2
* こんにゃく（個）: 2
* れんこん（本）: 2
* しいたけ（個）: 6
- ごぼう（本）: 1
- にんじん（本）: 1

■2人前のレシピの提案

豚汁（2人前）：数量不足(-)

- にんじん（本）: 1
- 豚バラ肉（g）: 200
- ごぼう（本）: 1
- さといも（個）: 4

ハンバーグ（2人前）：数量不足(-)

- ひき肉（g）: 150
- たまねぎ（個）: 1
- 牛乳（ml）: 100
- パン粉（g）: 20

＜プログラム＞

```
 1   import json
 2
 3   with open('recipes.json') as f:
 4       recipes = json.load(f)
 5   with open('stocks.json') as f:
 6       stocks = json.load(f)
 7
 8   def judge(ninzu, recipe_name, show_full=True):
 9       if (30):
10           print(f'「{recipe_name}」のレシピなし')
11           return
12
13       zairyou = recipes[recipe_name]
14       diff = (31)
15
16       if diff != set():
17           if not show_full:
18               return
19
20           print(f'{recipe_name}（{ninzu}人前）：在庫なし(*)，数量不足(-)')
21           for zai in diff:
22               print(f'  * {zai}: {ninzu * recipes[recipe_name][zai]}')
23           zaiko_ari = set(zairyou) - diff
24           for zai in zaiko_ari:
25               fu = ninzu * recipes[recipe_name][zai] - stocks[zai]
26               if fu > 0:
27                   print(f'  - {zai}: {fu}')
28       else:
29           fusoku = {}
30           for zai, suryou in recipes[recipe_name].items():
31               fu = (32)
32               if fu > 0:
33                   fusoku[zai] = fu
34
35           if len(fusoku) == 0:
36               print(f'{recipe_name}（{ninzu}人前）：調理可能')
37           else:
38               print(f'{recipe_name}（{ninzu}人前）：数量不足(-)')
39               for (33):
40                   print(f'  - {zai}: {fu}')
41
42   def suggest(ninzu):
43       print(f'■{ninzu}人前のレシピの提案')
44       for recipe in recipes:
45           (34)
```

```
46
47    judge(1, 'カレー')
48    judge(1, '豚汁')
49    judge(2, '筑前煮')
50    suggest(2)
```

(30) の解答群

ア　recipe_name in recipes
イ　recipe_name is not recipes
ウ　recipe_name is recipes
エ　recipe_name not in recipes

(31) の解答群

ア　set(stocks) - set(zairyou)
イ　set(zairyou) - set(stocks)
ウ　stocks - zairyou
エ　zairyou - stocks

(32) の解答群

ア　ninzu * suryou - stocks[zai]
イ　stocks[zai] - ninzu * suryou
ウ　stocks[zai] - suryou
エ　suryou - stocks[zai]

(33) の解答群

ア　zai, fu in fusoku.items()
イ　zai, fu in fusoku.keys()
ウ　zai, fu in fusoku.popitem()
エ　zai, fu in fusoku.values()

(34) の解答群

ア　judge(ninzu, recipe)
イ　judge(ninzu, recipe, False)
ウ　judge(ninzu, recipe, True)
エ　judge(ninzu, recipes)

問７　次のプログラムの説明およびプログラムを読んで，プログラム中の［　　］に入れる適切な字句を，解答群の中から選べ。

＜プログラムの説明＞

ソートアルゴリズムの一つであるバブルソートの処理時間を計測し，計測結果をグラフに出力するプログラムである。プログラムの処理時間は，実行時のコンピュータの状況によりばらつきがあると考えられるため，複数回実行し，処理時間の統計値（平均値や中央値など）を算出する。

バブルソートは，隣り合う要素の大小を比較しながら値を並べ替えるアルゴリズムであり，昇順に並べ替える場合は次の手順で行う。

○バブルソートの手順

① リストの先頭の要素と次の要素を比較し，先頭の要素が大きければ，その二つの要素の値を入れ替える。

② 比較する対象位置の二つをそれぞれ一つ後ろにずらし，①と同様の処理を行う。

③ ②の操作をリストの最後まで行うことにより，リストの右端の値が最大の値となる（1巡目）。

④ 比較するリストの範囲を一つ狭めて，① ② ③と同様に比較・交換を行う。

⑤ 比較する対象が先頭の二つの要素のみになるまで繰返し④を行うことで，リストの先頭から小さい順に値を並べ替えることができる（図１）。

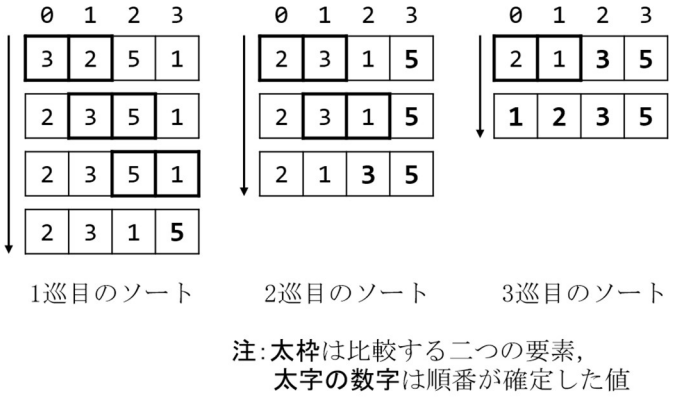


図１　長さ４のリストのバブルソートの例

このプログラムでは，0 ～ 99 までの整数から random モジュールの sample 関数でランダムに 80 個取り出したリストと 20 個取り出したリストを対象として，バブルソートの処理時間を指定回数測定し，最後に測定結果をグラフに出力する。

＜クラスの仕様＞

○BubbleSort クラス

バブルソートを用いた処理時間の計測や，その結果のグラフへの出力を行う。

- __init__ メソッド

インスタンスの初期化を行う。

- run メソッド

バブルソートの処理時間を指定回数計測し，結果を処理して表示する。

・処理手順

(1) time モジュールの perf_counter 関数で時刻を取得後，バブルソートを行い，再度時刻を取得し，前後の時刻の差分により秒単位の処理時間を算出する。

(2) (1)の処理を iter_count 回だけ実行し，各回の処理時間をミリ秒単位に変換してリストに格納する。

(3) (2)までの処理完了後，処理時間の中央値，計測処理の繰返し回数，入力したデータ（リスト）の長さ等の情報を出力する。このとき，中央値は numpy ライブラリの median 関数で算出し，小数第 3 位まで表示する。

(4) print_result が True であれば，ソート結果を出力する。

(5) 最後に，箱ひげ図を boxplot メソッドでプロットする。

- fig_show メソッド

Axes オブジェクトである_ax_bx にプロットした箱ひげ図のデータを，matplotlib ライブラリの show メソッドを実行してグラフに描画する。

BubbleSort クラスの仕様は，次のとおりである。

BubbleSortクラス	
iter_count	計測処理の繰返し回数。初期値は500。
num_list	ソート対象の数値を格納したリスト
print_result	ソート結果を表示するかどうかのフラグ。初期値はFalse。
_ax_bx	グラフの軸情報を表すAxesオブジェクト
_fig_num	描画するグラフの数
_proc_times	計測した各処理時間を格納するリスト
__init__メソッド	インスタンスの初期化を行うメソッドである。引数としてソート対象の数値のリストを受け取り，各インスタンス変数の初期化を行う。オブジェクト_ax_bxのsetメソッドは可変長引数をとり，y軸のスケールyscale，y軸のラベルylabelを設定する。
runメソッド	バブルソートの処理時間を指定回数計測し，結果を処理して表示するメソッドである。
fig_showメソッド	Axesオブジェクトである_ax_bxにプロットした箱ひげ図のデータを，matplotlibライブラリのshowメソッドを実行してグラフに描画するメソッドである。

このプログラムで使用するライブラリ，モジュール，メソッドおよび関数の説明を，次に示す。

matplotlibライブラリ
pyplotモジュール
subplotsメソッド
subplots(省略) 指定されたデータを基に図のオブジェクトと軸（または軸の配列）のオブジェクトを生成する。 引　数：省略（本問では引数を使用せず，デフォルト値を使用） 戻り値：図のオブジェクトと軸（または軸の配列）のオブジェクト
boxplotメソッド
boxplot(各種指定値) 指定されたデータを基に箱ひげ図を生成する。 引　数：設定する各種指定値 本問では，以下のプロパティを使用する。 x　箱ひげ図として描画する対象のデータ sym　外れ値の表示に関する設定を行う文字列。''は外れ値を非表示にする。 widths　ひげ図の幅 positions　箱ひげ図の位置。1から箱ひげ図の個数までの数 tick_labels　箱ひげ図の目盛りラベル 戻り値：箱ひげ図に関する辞書形式の情報（本問では使用しない）
showメソッド
show(省略) 指定されたデータを基にグラフを作成する。 引　数：省略（本問では引数を使用せず，デフォルト値を使用）

matplotlibライブラリ
axesモジュール
Axesクラス
setメソッド
set(各種指定値) 引数の各種指定値を受け取り，Axesオブジェクトのプロパティを一度に設定する。 引　数：設定する各種指定値 本問では，以下のプロパティを使用する。 yscale　Axesオブジェクトのy座標の単位を指定する。 ylabel　Axesオブジェクトのy座標のラベルを指定する。 戻り値：箱ひげ図に関する辞書形式の情報

numpyライブラリ
median関数
median(x, 省略) 引数で指定された入力データの中央値を返す。 引　数：本問では，以下の引数を使用する。 　　x　　配列または配列に変換可能なオブジェクト 戻り値：中央値

timeモジュール
perf_counter関数
perf_counter() パフォーマンスカウンターの値（短時間計測の高精度な秒数）を返す。 戻り値：パフォーマンスカウンターの値

randomモジュール
sample関数
sample(population, k, 省略) 指定されたデータを基にデータサンプリング（ランダムに選択）する。 引　数：本問では，以下の引数を使用する。 　　population　サンプリングの元データ 　　k　　サンプリングの個数 戻り値：サンプリングしたリスト

≪実行結果例≫

```
=== No.1: Result ===

  Time (Median): 0.365[ms]

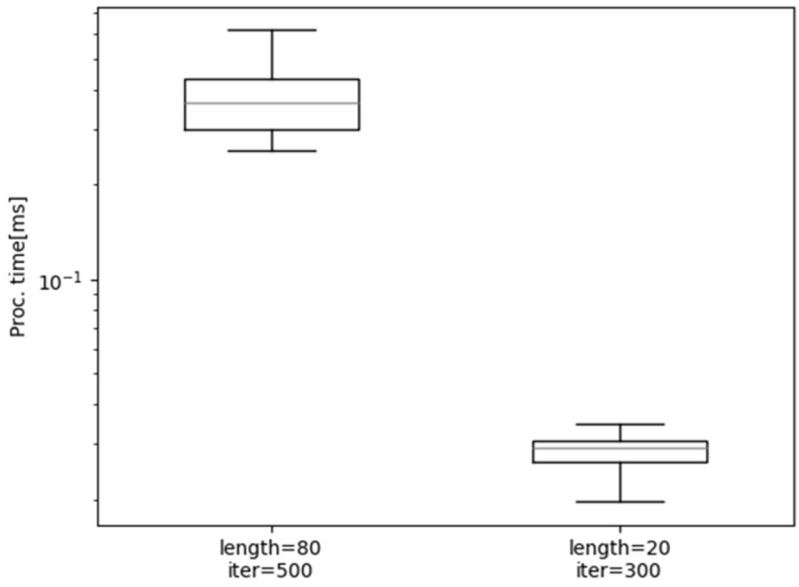
  Iter.: 500, Length: 80

=== No.2: Result ===

  Time (Median): 0.029[ms]

  Iter.: 300, Length: 20

  Sort result: [1, 2, 4, 10, 18, 25, 30, 32, 41, 56, 57, 59, 77, 80, 90, 91,
  92, 93, 97, 99]
```



＜プログラム＞

```
 1   import matplotlib.pyplot as plt
 2   import numpy as np
 3   import time
 4   from random import Random
 5
 6   class BubbleSort:
 7       def __init__(self, num_list):
 8           self.iter_count = 500
 9           self.num_list = num_list
10           self.print_result = False
11           _, self._ax_bx = plt.subplots()
12           self._fig_num = 0
13           self._proc_times = []
14           kwargs = dict(yscale='log', ylabel='Proc. time[ms]')
15           self._ax_bx.set(    (35)    )
16
17       def run(self):
18           self._proc_times = []
19           for _ in range(self.iter_count):
20               li = self.num_list.copy()
21               n = len(li)
22               start = time.perf_counter()
23               for i in range(n - 1, 0, -1):
24                   for j in range(0, i, 1):
25                       if li[j] > li[j + 1]:
26                           (36)
27               end = time.perf_counter()
28               (37)
29           self._fig_num += 1
30
31           print(f'=== No.{self._fig_num}: Result ===')
32           print(f' Time (Median): {    (38)    }[ms]')
33           print(f' Iter.:{self.iter_count}, Length: {len(self.num_list)}')
34
35           self._ax_bx.boxplot(self._proc_times,
36               sym='', widths=0.5, positions=[self._fig_num],
37               tick_labels=[
38                   f'length={len(self.num_list)}¥niter={self.iter_count}'])
39
40           if self.print_result:
41               print(f'  Sort result: {li}')
42
43       def fig_show(self):
44           plt.show()
45
```

```
46    rd = Random()
47    num_list = rd.sample([x for x in range(100)], 80)
48    bs = (39)
49    bs.run()
50
51    num_list = rd.sample([x for x in range(100)], 20)
52    bs.num_list = num_list
53    bs.iter_count = 300
54    bs.print_result = True
55    bs.run()
56
57    bs.fig_show()
```

(35) の解答群

ア kwargs
イ *kwargs
ウ **kwargs
エ ***kwargs

(36) の解答群

ア li[j], li[j] = li[j], li[j]
イ li[j], li[j + 1] = li[j + 1], li[j]
ウ li[j], li[j - 1] = li[j - 1], li[j]
エ li[j - 1], li[j + 1] = li[j + 1], li[j - 1]

(37) の解答群

ア self._proc_times.append(end - start)
イ self._proc_times.append((end - start) * 1000)
ウ self._proc_times.append((end - start) / 1000)
エ self._proc_times.append((end - start) // 1000)

(38) の解答群

ア median(self._proc_times):.3d
イ median(self._proc_times):.3f
ウ np.median(self._proc_times):.3d
エ np.median(self._proc_times):.3f

(39) の解答群

ア　BubbleSort()
イ　BubbleSort(num_list)
ウ　BubbleSort(None, num_list)
エ　BubbleSort(self, num_list)

試験問題は著作権法上の保護を受けています。
試験問題の一部または全部について，サーティファイから文書による許諾を得ずに，いかなる方法においても私的使用の範囲を超えて，無断で複写，複製することを禁じます。
無断複製，転載は損害賠償，著作権法の罰則の対象になることがあります。
©CERTIFY Inc.2024

試験問題に記載されている会社名又は製品名は，それぞれ各社の商標又は登録商標です。
なお，試験問題では，® 及び ™ を明記していません。